

Investigating the Middle-Square Random Number Generator

Pseudo-Random Number Generation • Chi-Square Goodness-of-Fit Test

In this assignment, you will implement a pseudo-random number generator using the **Middle-Square Method**, investigate its weaknesses, and statistically evaluate the generated numbers using a **Chi-Square goodness-of-fit test**.

Note: Do not use Python's `random` module.

Task 1 — 1

Start with a **4-digit integer seed** X_0 . For every iteration:

1. Square the current value.
2. Represent the result as an **8-digit number**, adding leading zeros if necessary.
3. Extract the **middle four digits**.
4. Use these four digits as the **next value**.

Example

Seed = 5731

$$5731^2 = 32,844,361 \implies \text{Middle four digits} = \mathbf{8443}$$

$$8443^2 = 71,284,249 \implies \text{Middle four digits} = \mathbf{2842}$$

Implement the following function and generate **at least 100 values**:

```
1 def middle_square(seed, n):
2     # your implementation here
3     pass
```

Task 2 — 1

Note

The Middle-Square method has an important weakness. Find **at least one seed** that causes the generator to **repeatedly generate the same value**.

For the problematic seed, fill in the following report:

Edge Case Report

Field	Value
Seed	
First repeated value	
Iteration at which it repeats	
Cycle length	

Task 3 — 1

Generating numbers that *appear* random is not sufficient. Use a **Chi-Square goodness-of-fit test** to investigate whether your generated numbers are approximately **uniformly distributed** over $[0, 1)$.

1 Binning Scheme Divide the interval $[0, 1)$ into **10 equal-sized bins**:

Bin	Interval
Bin 1	$[0.0, 0.1)$
Bin 2	$[0.1, 0.2)$
$\vdots$	$\vdots$
Bin 10	$[0.9, 1.0)$

Generate **1 000 random numbers** and count how many values fall into each bin. Under the assumption of a uniform distribution, each bin has an expected count of:

$$E = \frac{1000}{10} = 100$$

1 Hypothesis & Decision Rule

- **Null Hypothesis H_0 :** The generated numbers are uniformly distributed over $[0, 1)$.
- **Significance level:** $\alpha = 0.05$
- **Decision rule:** If $p < 0.05$, reject H_0 ; otherwise, do not reject.

You may use Python's statistical libraries (e.g. `scipy.stats`) to perform the test.

1 Results Table Perform the Chi-Square test for the following **three seeds**, generating 1 000 values each time:

Seed	χ^2 statistic	p-value	Decision
5731			
6239			
Problematic Seed			

Task 4 — 1

Reflection Question

Does **passing the Chi-Square test** necessarily mean that the generator is a **good random-number generator**? Explain briefly.

End of Assignment